BASICS OF GenAI

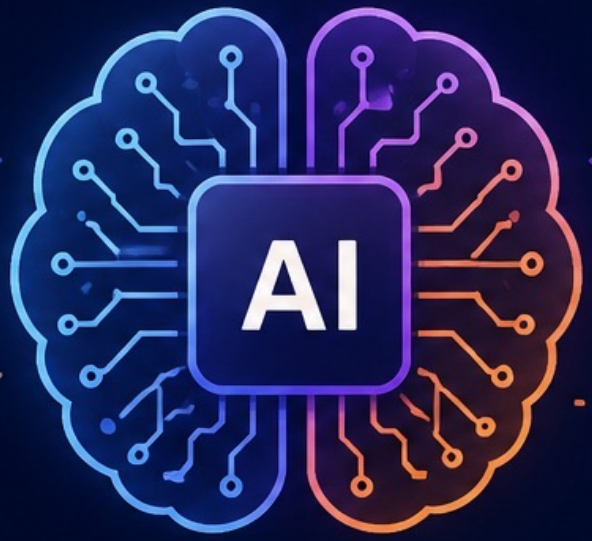
Understand. Create. Innovate.



TEXT GENERATION



IMAGE GENERATION



CODE GENERATION



IDEAS TO IMPACT





UNDERSTAND



CREATE



INNOVATE



EMPOWER

Lecture: Getting Started with Text Generation

Agenda

1. Environment & API Setup
2. Generation Models vs Reasoning Models
3. Base vs Instruct vs Chat Models
4. Chat Templates
5. Decoding & Generation Strategies

Part 2: Understanding Model Types

2. Base vs Instruct vs Chat Models

Not all LLMs behave the same way—their training determines their capabilities.

Understanding these differences is critical for choosing the right model.

	BASE MODEL	INSTRUCT MODEL	CHAT MODEL
Training	Next-token prediction on raw text (unsupervised)	Fine-tuned on instruction-response pairs (supervised)	Fine-tuned on multi-turn conversations (supervised)
Behavior	Continues text, doesn't follow instructions **Prompt:** Once upon a time **Output:** ... *there was a king who lived in a small village. He loved...*	Follows single-turn instructions **Instruction:** Summarize this article. **Output:** *This article discusses the benefits of renewable energy...*	Handles dialogue, maintains context across turns **User:** What is quantum computing? **Assistant:** Quantum computing uses qubits to perform computations... **User:** Can you give an example? **Assistant:** Sure! One example is using qubits to factor large numbers...
Use Case	Text completion, fine-tuning starting point	Q&A, summarization, single tasks	Chatbots, assistants, conversations

Key Takeaway: The more a model is trained on conversations and instructions, the better it becomes at understanding and responding to real-world user needs.

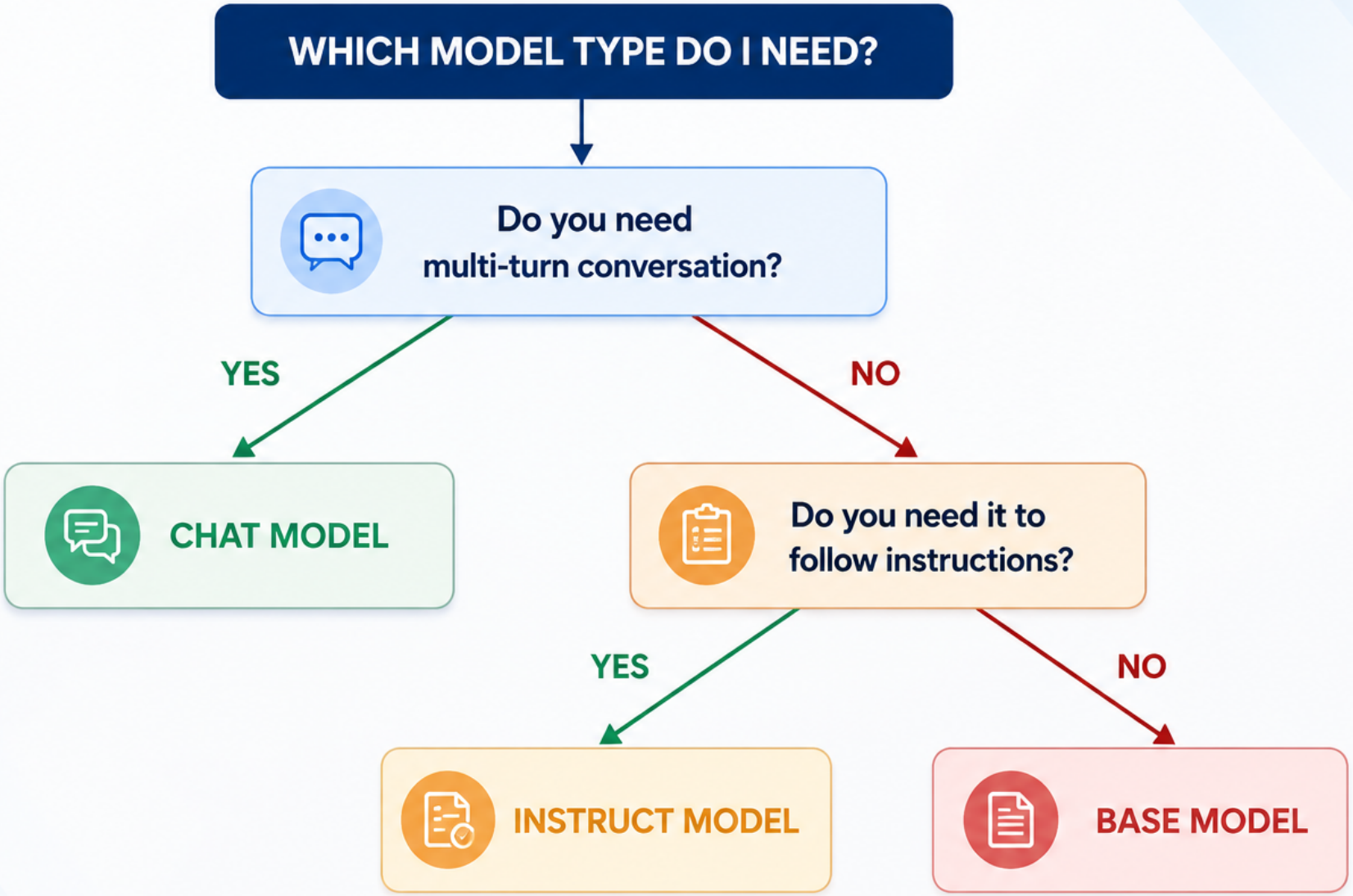
7 Key Differences Summary

Aspect	Base Model	Instruct Model	Chat Model
Training	Next-token prediction	Instruction fine-tuning	Conversation fine-tuning
Input Format	Raw text	Instruction template	Message list with roles
Output	Text continuation	Task completion	Conversational response
Multi-turn	✗ No context awareness	⚠ Limited	✓ Full context
System Prompt	✗ Not understood	⚠ Sometimes	✓ Fully supported
When to Use	Fine-tuning base	Single-turn tasks	Assistants, chatbots

Key Takeaway

As models are fine-tuned from base to instruct to chat,
their ability to understand instructions and maintain conversation context increases significantly.

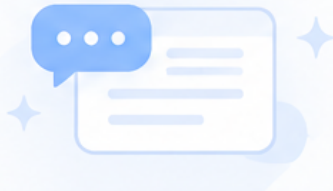
Which Model Type Do I Need?



14 Chat Template Comparison Table

Model	System Support	BOS Token	EOS / Turn Token	Thinking Mode
Llama 3.x	Yes	`<|begin_of_text|>`	`<|eot_id|>`	No
DeepSeek V3.1	Yes	`<|begin_of_sentence|>`	`<|end_of_sentence|>`	Yes
Qwen 2.5	Yes	`<|im_start|>`	`<|im_end|>`	No
Qwen 3	Yes	`<|im_start|>`	`<|im_end|>`	Yes
Mistral	Workaround	`<s>`	`</s>`	No
Phi-3 / Phi-4	Yes	`<|system|>`	`<|end|>`	No
Gemma 2	Workaround	`<start_of_turn>`	`<end_of_turn>`	No

Key Takeaway: Different models use different special tokens and system support mechanisms.
Always use the appropriate chat template for best results and compatibility.



SUMMARY

Temperature in Softmax: Controlling Model Confidence & Creativity

KEY TAKEAWAYS

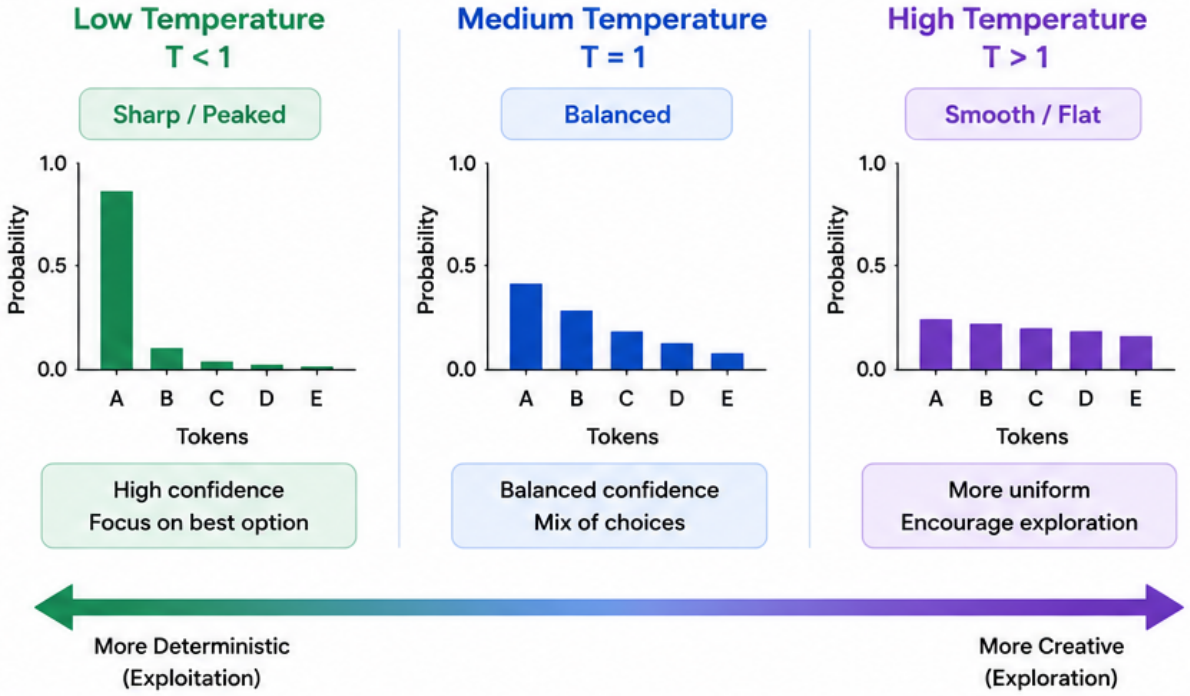
1. **Temperature in softmax** controls the "sharpness" of probability distributions.

2. **Low temperature (T < 1):**
   - Sharp, peaked distributions
   - High confidence predictions
   - **Useful for:** greedy decoding, deterministic outputs, exploitation

3. **High temperature (T > 1):**
   - Smooth, flat distributions
   - More uniform probabilities
   - **Useful for:** exploration, knowledge distillation, creative generation

4. **Applications:**
   - **Language models:** Control creativity in text generation
   - **Knowledge distillation:** Preserve class relationships
   - **Reinforcement learning:** Balance exploration vs exploitation
   - **Calibration:** Adjust model confidence

EFFECT OF TEMPERATURE ON SOFTMAX DISTRIBUTIONS



FORMULA

$$\text{softmax}_T(z_i) = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}$$

where:

- z_i = logit for token i
- n = total number of tokens
- T = temperature parameter

Bottom Line:

Temperature (T) is a simple yet powerful knob that lets us control the trade-off between **confidence** and **creativity**, **determinism** and **diversity** in model outputs.

SUMMARY

Temperature in Softmax: Controlling Model Confidence & Creativity

KEY TAKEAWAYS

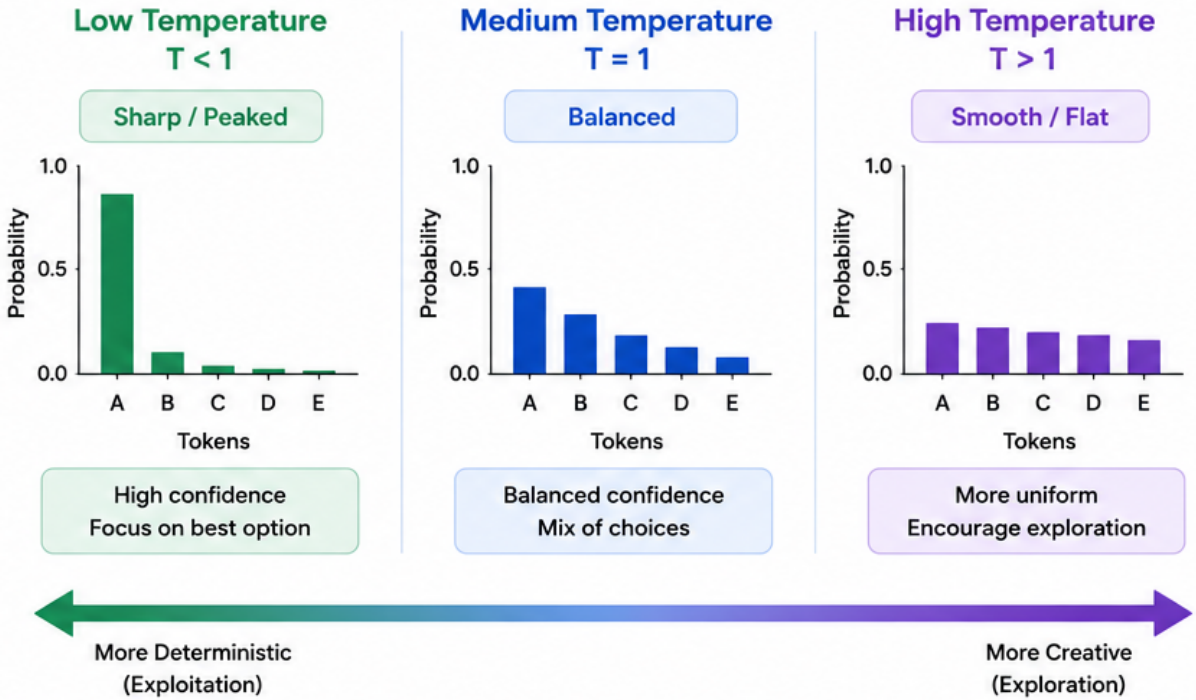
1. **Temperature in softmax** controls the "sharpness" of probability distributions.

2. **Low temperature (T < 1):**
   - Sharp, peaked distributions
   - High confidence predictions
   - **Useful for:** greedy decoding, deterministic outputs, exploitation

3. **High temperature (T > 1):**
   - Smooth, flat distributions
   - More uniform probabilities
   - **Useful for:** exploration, knowledge distillation, creative generation

4. **Applications:**
   - **Language models:** Control creativity in text generation
   - **Knowledge distillation:** Preserve class relationships
   - **Reinforcement learning:** Balance exploration vs exploitation
   - **Calibration:** Adjust model confidence

EFFECT OF TEMPERATURE ON SOFTMAX DISTRIBUTIONS



FORMULA

$$\text{softmax}_T(z_i) = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}$$

where:

- z_i = logit for token i
- n = total number of tokens
- T = temperature parameter

Bottom Line:

Temperature (T) is a simple yet powerful knob that lets us control the trade-off between **confidence** and **creativity**, **determinism** and **diversity** in model outputs.

METHOD 1: GREEDY SEARCH

OVERVIEW

Greedy Search is the simplest decoding strategy:

- At each step, select the token with the **highest probability**
- No backtracking or reconsideration

ALGORITHM

1. Start with input tokens
2. While not done:
   a. Get probability distribution for next token
   b. Select token with **argmax**(probability)
   c. Append token to sequence
3. Return sequence

PROS

- Fast and efficient
- Deterministic (same input = same output)
- Low memory footprint

CONS

- **Myopic:** doesn't consider long-term consequences
- Can get stuck in repetitive patterns
- May miss better overall sequences

HOW IT WORKS (EXAMPLE)



Key Takeaway: Greedy Search always picks the most likely next token at each step. It is **fast and simple**, but may **not always** produce the best overall sequence.

METHOD 2: BEAM SEARCH

OVERVIEW

Beam Search maintains multiple hypotheses (beams) simultaneously:

- At each step, consider the top n tokens (where n = beam width)
- Keep track of n best sequences based on cumulative score
- After generation completes, return the sequence with highest overall score

SCORING FUNCTION

$$score(w) = \frac{1}{|w|} \sum_{i=1}^{|w|} \log P(w_i \mid w_1, \ldots, w_{i-1})$$

We use:

- ✓ **Log probabilities** (easier to compute, avoid underflow)
- ✓ **Length normalization** (prevent bias toward shorter sequences)

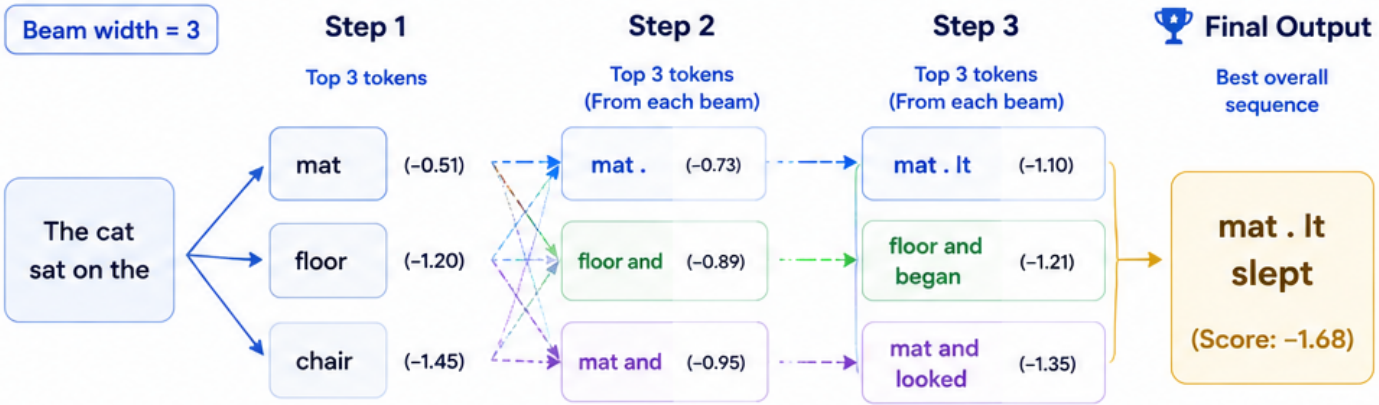
PROS

- Considers multiple paths simultaneously
- Better at finding high-quality sequences
- More reliable than greedy for translation, summarization

CONS

- More computationally expensive (O(beam_width))
- Still can produce generic/repetitive text
- May not be diverse enough for creative tasks

HOW BEAM SEARCH WORKS (EXAMPLE)



KEY TAKEAWAY: Beam Search looks ahead by keeping multiple strong candidates at each step, leading to **better quality outputs** than greedy search, at the cost of more computation.

METHOD 3: TOP-K SAMPLING

OVERVIEW

Top-k Sampling introduces controlled randomness:

1. Consider only the top k most likely tokens
2. Randomly sample from these k tokens based on their probabilities
3. Ignore all other tokens (set their probability to 0)

PROS

- More diverse outputs than greedy/beam search
- Simple to implement and understand
- Good for creative tasks (story generation, brainstorming)

CONS

- Fixed k doesn't adapt to situation
- May still include low-quality tokens
- Choosing k is task-dependent

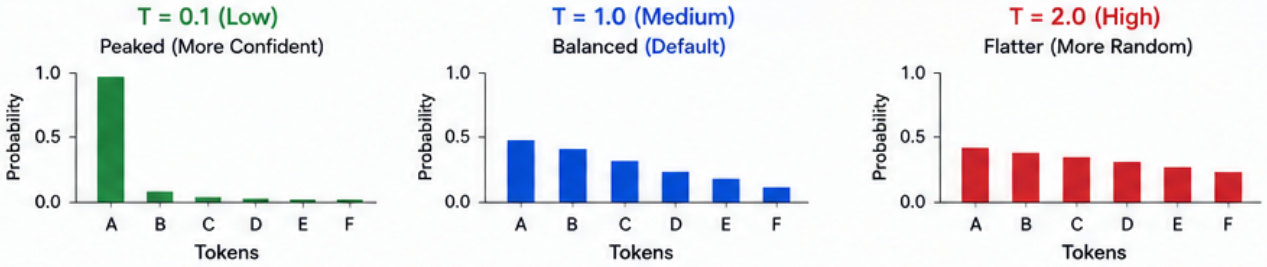
THE ROLE OF TEMPERATURE

Temperature T controls randomness:

$$P(w_i) = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

Where z_i are the logits.

- **T = 1.0:** Standard softmax (default)
- **T < 1.0 (e.g., 0.1):** More confident, peaked distribution
- **T > 1.0 (e.g., 2.0):** Flatter, more random distribution



HOW TOP-K SAMPLING WORKS (EXAMPLE)

Step 1: Raw Probabilities → Step 2: Keep Top-k (k = 5) → Step 3: Renormalize → Step 4: Sample One Token

Token	Probability
the	0.28
a	0.18
an	0.12
this	0.08
that	0.06
many	0.03
some	0.02
other	0.01
...	

Token	Probability
the	0.28
a	0.18
an	0.12
this	0.08
that	0.06

Others set to 0

Token	Renormalized Probability
the	0.40
a	0.26
an	0.17
this	0.11
that	0.06

Sum = 1.00

Randomly sample based on renormalized probs



Example outcome: "a"

KEY TAKEAWAY

Top-k Sampling adds controlled randomness by sampling from the top k likely tokens. It improves diversity over greedy/beam search while remaining simple and efficient.

METHOD 4: NUCLEUS (TOP-P) SAMPLING

OVERVIEW

Nucleus Sampling (also called **top-p sampling**) is more adaptive:

1. Sort tokens by probability (descending)
2. Keep adding tokens until cumulative probability exceeds threshold p
3. Sample from this dynamic **"nucleus"** of tokens

ALGORITHM

```
Sort tokens by P(token) descending
cumsum = 0
nucleus = []
for token in sorted_tokens:
    cumsum += P(token)
    nucleus.append(token)
    if cumsum >= p:
        break
Sample from nucleus
```

KEY DIFFERENCE FROM TOP-K

Top-k Sampling	Top-p Sampling (Nucleus)
Fixed number of tokens (k) regardless of their probabilities	Variable number of tokens based on cumulative probability (p)

EXAMPLES

With p = 0.9

▶ If top token has 95% probability → nucleus = 1 token

Token	Probability	Cumulative
A	0.95	0.95
B	0.03	0.98
C	0.01	0.99
D	0.005	0.995
E	0.005	1.000

With p = 0.9

If probabilities are [30%, 25%, 20%, 15%, 10%] → nucleus = 4 tokens

Token	Probability	Cumulative
A	0.30	0.30
B	0.25	0.55
C	0.20	0.75
D	0.15	0.90
E	0.10	1.00

Nucleus size **adapts** to the model's **confidence**: smaller when confident, larger when uncertain.

PROS

- Dynamically adapts to confidence level
- More robust across different contexts
- Better quality-diversity tradeoff
- State-of-the-art for many generation tasks

CONS

- Slightly more complex to implement
- p parameter still needs tuning

KEY TAKEAWAY

Nucleus (Top-p) Sampling selects a **dynamic** set of tokens whose cumulative probability exceeds p and samples from it. It provides a better balance between **quality and diversity** and is widely used in modern LLM generation.

SUMMARY & BEST PRACTICES

QUICK DECISION GUIDE

Task Type	Best Strategy	Typical Parameters
Translation	Beam Search	num_beams = 5
Summarization	Beam Search	num_beams = 4, length_penalty = 1.0
Question Answering	Greedy or Beam	num_beams = 1–3
Creative Writing	Top-*k* or Nucleus	*k* = 50, *p* = 0.95, T = 0.8–1.0
Code Generation	Nucleus	*p* = 0.95, T = 0.2–0.5
Chatbots	Nucleus	*p* = 0.9, T = 0.7
Data-to-Text	Beam Search	num_beams = 3–5

PARAMETER GUIDELINES

Temperature (T)

- **0.1–0.5:** Very focused, deterministic outputs
- **0.5–1.0:** Balanced creativity and coherence ✅ (Recommended default)
- **1.0–2.0:** More creative, diverse, but potentially less coherent

Top-*k* (*k*)

- ***k* = 10–20:** Conservative, coherent
- ***k* = 30–50:** Moderate diversity ✅ (Recommended default)
- ***k* = 50+:** High diversity, more creative

Top-*p* (*p*)

- ***p* = 0.8–0.9:** More focused ✅ (Good for factual tasks)
- ***p* = 0.92–0.95:** Balanced ✅ (Recommended default)
- ***p* = 0.95–0.99:** More diverse ✅ (Good for creative tasks)

Beam Width (beams)

- **beams = 1:** Greedy search
- **beams = 3–5:** Good tradeoff ✅ (Recommended)
- **beams = 5–10:** Better quality but slower
- **beams = 10+:** Diminishing returns

KEY INSIGHTS

1. **Greedy Search:** Fast but myopic – use when speed matters more than quality
2. **Beam Search:** Best for tasks with "correct" answers (translation, summarization)
3. **Top-*k* Sampling:** Simple randomness – good baseline for creative tasks
4. **Nucleus Sampling:** Most versatile – adapts to model confidence ✅ Recommended default
5. **Combining Strategies:** You can use temperature with any sampling method!

COMMON PITFALLS

DON'T:

- ❌ Use high temperature with greedy/beam search (they don't sample)
- ❌ Set both top-*k* and top-*p* (redundant – pick one)
- ❌ Use very high temperature (>1.5) without testing
- ❌ Ignore length penalties in beam search for long generations

BEST PRACTICES

DO:

- ✅ Experiment with different strategies for your specific task
- ✅ Start with nucleus sampling (*p*=0.92, T=0.8) as baseline
- ✅ Use beam search for deterministic, factual tasks
- ✅ Monitor both quality and diversity in outputs
- ✅ Consider combining multiple generations and reranking

BOTTOM LINE: Choose the decoding strategy that best balances **quality**, **diversity**, and **efficiency** for your specific use case.